CHAITANYA BHARATHI INSTITUTE OF TECHNOLOGY

Department of Information Technology

Hyderabad, Telangana, India

FragmentLab

Interactive IP Fragmentation & MTU Discovery

A Web-Based Simulator for IPv4 Packet Fragmentation, Reassembly, and Path MTU Discovery

Computer Networks Laboratory Project

Submitted By

Veer Nala Shanmukhavarun

Department of Information Technology

Chaitanya Bharathi Institute of Technology

October 2026

I. ABSTRACT

Understanding how IP packets are fragmented and reassembled is fundamental to mastering computer networking. When a packet exceeds the Maximum Transmission Unit (MTU) of a network link, routers must fragment it into smaller pieces that fit within the link constraints. This process introduces header overhead, requires careful offset tracking, and demands successful reassembly at the destination. FragmentLab is an interactive, web-based simulator that provides hands-on visualization of IPv4 packet fragmentation, fragment reassembly, and Path MTU Discovery (PMTUD). Built with React and modern JavaScript, the application implements RFC 791 fragmentation calculations and allows users to configure packet sizes, IP header sizes, and MTU values for multiple network links. The simulator demonstrates core networking concepts including fragment offsets (in 8-byte units), the More Fragments (MF) flag, Identification field grouping, Don't Fragment (DF) behavior, ICMP Fragmentation Needed messages (Type 3, Code 4), and the iterative Path MTU Discovery process. A comparison mode enables side-by-side evaluation of three approaches: normal fragmentation, Don't Fragment packet dropping, and PMTUD. The tool also simulates lost fragments and failed reassembly scenarios, providing comprehensive coverage of fragmentation-related networking concepts.

Keywords: *IPv4 f*

II. INTRODUCTION

The Internet Protocol version 4 (IPv4), defined in RFC 791, is the foundational protocol of the Internet. One of its key mechanisms is fragmentation - the ability to split a datagram into smaller fragments when the packet size exceeds the Maximum Transmission Unit (MTU) of a network link. While modern networks increasingly rely on Path MTU Discovery to avoid fragmentation altogether, understanding the fragmentation process remains essential for network engineers, security analysts, and students of computer networking.

FragmentLab addresses the gap in interactive networking education tools by providing a visual, hands-on simulator specifically focused on MTU constraints, fragmentation, and reassembly. Unlike generic packet-flow animation tools or route simulators, FragmentLab concentrates exclusively on how packets are sized, split into fragments, transmitted, and reconstructed - making it a targeted educational resource for these concepts.

A. Motivation

Students and professionals learning about IP fragmentation often struggle to visualize how a large packet is broken down, what header fields change across fragments, and how the receiver reassembles the original datagram. Key concepts such as 8-byte offset alignment, the interaction between More Fragments (MF) and Don't Fragment (DF) flags, and the role of ICMP in Path MTU Discovery are difficult to grasp from textbooks alone. FragmentLab bridges this gap with interactive, real-time visualization.

B. Objectives

- Implement accurate IPv4 fragmentation calculations following RFC 791 specifications
- Visualize the fragmentation process with proportional packet and fragment bars
- Display all relevant IPv4 header fields: fragment offsets, identification, MF/DF flags
- Simulate multi-link fragmentation through paths with varying MTU sizes
- Demonstrate reassembly with selectable lost fragment simulation
- Compare three approaches: normal fragmentation, DF bit dropping, and Path MTU Discovery

- Provide a PMTUD simulation showing ICMP Fragmentation Needed message flow

III. THEORETICAL BACKGROUND

A. IPv4 Header Structure

The IPv4 header is a minimum of 20 bytes (without options) and a maximum of 60 bytes (with options). Key fields relevant to fragmentation include:

- Total Length (16 bits): The total datagram size in bytes, including header and payload.
- Identification (16 bits): A unique value assigned by the sender to identify fragments belonging to the same original datagram.
- Flags (3 bits): Bit 0 is reserved. Bit 1 is Don't Fragment (DF). Bit 2 is More Fragments (MF).
- Fragment Offset (13 bits): The offset of the fragment's payload relative to the beginning of the original datagram, measured in 8-byte units.
- Time to Live (TTL): Decremented at each hop; prevents infinite routing loops.
- Protocol (8 bits): Identifies the upper-layer protocol (TCP=6, UDP=17, ICMP=1).

B. Maximum Transmission Unit (MTU)

The MTU is the largest packet size (in bytes) that a network link can transmit in a single frame. Common MTU values include: Ethernet (1500 bytes), PPPoE (1492 bytes), VPN tunnels (1300-1400 bytes), and Jumbo Frames (9000 bytes). When a router receives a packet larger than the outgoing link's MTU, it must either fragment the packet or drop it (if the DF bit is set).

C. IPv4 Fragmentation Process

When a router must fragment a packet, it performs the following steps:

1. Calculate the maximum payload per fragment: floor((MTU - header_size) / 8) x 8 bytes. The payload must be a multiple of 8 bytes (except for the last fragment).

2. Copy the IP header to each fragment, updating: Total Length, Fragment Offset, and the More Fragments (MF) flag.

3. The Identification field is copied unchanged to all fragments, enabling the receiver to group them.

4. Each fragment except the last has MF=1. The last fragment has MF=0 (or inherits the original MF value).

5. Fragment offsets are expressed in 8-byte units in the IP header field.

D. Fragment Reassembly

The destination host reassembles fragments using the Identification field to group related fragments, the Fragment Offset to order them, and the MF flag to detect the final fragment. If any fragment is lost, reassembly fails and the entire datagram must be retransmitted. This is a significant vulnerability - the loss of a single fragment wastes the bandwidth consumed by all other fragments.

E. Path MTU Discovery (PMTUD)

Path MTU Discovery (RFC 1191) is a technique to determine the smallest MTU along a network path without actually

fragmenting packets. The process works as follows:

1. The sender sets the Don't Fragment (DF) bit on all outgoing packets.

2. If a router cannot forward the packet without fragmentation, it drops the packet and sends an ICMP "Destination Unreachable - Fragmentation Needed and DF Set" message (Type 3, Code 4) back to the sender.

3. The ICMP message includes the Next-Hop MTU of the link that caused the failure.

4. The sender reduces its packet size to match the reported MTU and retransmits.

5. This process repeats until the packet can traverse the entire path without fragmentation.

IV. SYSTEM DESIGN & ARCHITECTURE

A. Technology Stack

FragmentLab is built using the following technologies:

- Frontend Framework: React 19 with Vite 8 build tool
- Language: JavaScript (ES2022+) with JSX syntax
- Styling: Vanilla CSS with CSS custom properties (design tokens)
- Typography: Inter (sans-serif) and JetBrains Mono (monospace) from Google Fonts
- Visualization: HTML/CSS-based proportional bar charts and SVG network path diagrams
- Build: Vite development server with Hot Module Replacement (HMR)

B. Project Structure

```
fragmentlab/
  index.html              # Entry point with Google Fonts
  package.json            # Dependencies and scripts
  vite.config.js          # Vite configuration
  src/
    main.jsx              # React DOM render entry
    index.css             # Global design system (700+ lines)
    App.jsx               # Main app with state management
    engine/
      fragmentation.js    # Core fragmentation engine (RFC 791)
    components/
      ConfigPanel.jsx     # Packet & link configuration UI
      PacketVisualizer.jsx # Visual fragment bars & MTU markers
      FragmentTable.jsx   # Detailed fragment header table
      ReassemblyView.jsx  # Reassembly simulation with lost frags
      ComparisonMode.jsx  # Side-by-side approach comparison
      PMTUDView.jsx       # Path MTU Discovery visualization
      StatsBar.jsx        # Summary statistics dashboard
```

C. Core Engine: fragmentation.js

The fragmentation engine implements the following core functions:

- createPacket(): Creates an IPv4 packet object with configurable totalSize, headerSize, protocol, TTL, DF bit, and source/destination IPs.
- fragmentPacket(packet, mtu): Fragments a single packet based on the given MTU. Calculates 8-byte-aligned payload sizes, fragment offsets, and MF flags. Returns fragments or ICMP error if DF is set.
- fragmentThroughPath(packet, links): Iteratively fragments packets through multiple links with different MTUs, tracking the fragmentation state at each hop.
- reassembleFragments(fragments, lostIndices): Simulates reassembly by checking for gaps, ordering by offset, and verifying the presence of the final fragment (MF=0).
- simulatePMTUD(packet, links): Simulates Path MTU Discovery by iterating through links, detecting oversized packets, generating ICMP messages, and discovering the path MTU.

D. Fragmentation Algorithm

The core fragmentation algorithm (RFC 791 compliant):

```
function fragmentPacket(packet, mtu) {
  // Max payload per fragment (must be multiple of 8)
  maxPayload = floor((mtu - headerSize) / 8) * 8;

  remainingPayload = packet.payloadSize;
  currentOffset = 0;  // in bytes

  while (remainingPayload > 0) {
    isLast = remainingPayload <= maxPayload;
    fragPayload = isLast ? remainingPayload : maxPayload;

    fragment = {
      totalSize: fragPayload + headerSize,
      fragmentOffset: currentOffset,
      offsetField: currentOffset / 8,  // 8-byte units
      moreFragments: !isLast,
      identification: packet.identification
    };

    currentOffset += fragPayload;
    remainingPayload -= fragPayload;
  }
}
```

V. DEMO SCENARIO: 4000-BYTE PACKET

Demo Configuration
Packet: 4,000 bytes (20B header + 3,980B payload) | Link: Ethernet | MTU: 1,500 bytes | Protocol: TCP

A. Fragmentation Calculation

Given a 4,000-byte IPv4 packet (20-byte header + 3,980-byte payload) traversing an Ethernet link with a 1,500-byte MTU:

1. Maximum payload per fragment = floor((1500 - 20) / 8) x 8 = floor(1480 / 8) x 8 = 185 x 8 = 1,480 bytes

2. Number of fragments = ceil(3980 / 1480) = ceil(2.689) = 3 fragments

B. Fragment Details

#	Total (B)	Header (B)	Payload (B)	Identification	Offset (B)	Offset Field	MF
0	1500	20	1480	0x0001	0	0	MF=1
1	1500	20	1480	0x0001	1480	185	MF=1
2	1040	20	1020	0x0001	2960	370	MF=0

C. Analysis

Key observations from the demo scenario:

1. Header Overhead: The original packet had one 20-byte header. After fragmentation, three headers are transmitted (3 x 20 = 60 bytes), adding 40 bytes of overhead.

2. Total Bytes Transmitted: 1500 + 1500 + 1040 = 4,040 bytes vs. the original 4,000 bytes.

3. Fragment Offset Alignment: All offsets are multiples of 8 (0, 1480, 2960). The offset field stores values in 8-byte units (0, 185, 370).

4. More Fragments Flag: Fragments #0 and #1 have MF=1. Fragment #2 (the last) has MF=0, signaling the end of the fragment chain.

5. Identification: All three fragments share the same Identification value (0x0001), allowing the receiver to group them for reassembly.

VI. APPLICATION FEATURES

A. Fragmentation Mode

The primary mode of FragmentLab. Users configure:
- Total Packet Size (68 - 65,535 bytes) via slider and numeric input
- IP Header Size (20 - 60 bytes, in 4-byte increments for IP options)
- Don't Fragment (DF) bit toggle
- Protocol selection (TCP, UDP, ICMP, GRE)
- TTL value (1 - 255)
- Multiple network links with configurable names and MTUs

The visualization shows the original packet as a proportional bar (header + payload), a red MTU marker line, and each resulting fragment with its own header/payload bar, offset, and MF flag. A detailed table displays all IPv4 header fields for each fragment.

B. Reassembly Simulation Mode

This mode allows users to simulate the reassembly process at the destination. Fragment chips are displayed below the visualization - clicking a chip marks that fragment as "lost." The reassembly buffer visualizes which portions of the original payload have been received (colored segments) and which are missing (hatched gaps). Statistics show received/total fragments, bytes received, and gap count. If any fragment is missing, the reassembly fails with a descriptive error message.

C. Comparison Mode

Side-by-side comparison of three approaches to handling oversized packets:

1. Normal Fragmentation (DF=0): The router fragments the packet. Shows fragment count, total bytes transmitted, and header overhead.

2. Don't Fragment (DF=1): The router drops the packet and sends ICMP Type 3, Code 4. Shows the ICMP message details and reported Next-Hop MTU.

3. Path MTU Discovery: The sender iteratively discovers the path MTU using DF+ICMP feedback. Shows the discovered path MTU, adjusted packet size, and number of ICMP messages received.

D. Path MTU Discovery (PMTUD) Mode

A dedicated visualization of the PMTUD process. Features include:
- An SVG network path diagram showing source, each link (with MTU labels), and destination
- Bottleneck link detection and highlighting
- Step-by-step discovery process with ICMP message details
- Summary statistics (original size, path MTU, ICMP messages, link count)
- Educational explanation of how PMTUD works

E. Multi-Hop Fragmentation

Users can add multiple network links with different MTUs to simulate real-world paths. For example, a path through Ethernet (MTU 1500) -> VPN Tunnel (MTU 1300) -> PPPoE (MTU 1492) demonstrates how fragments may be re-fragmented at intermediate hops, progressively increasing the fragment count and header overhead.

VII. NETWORKING CONCEPTS COVERED

IPv4 Header Fields:	Total Length, Identification, Flags (DF, MF), Fragment Offset, TTL, Protocol, Source/Destination IP addresses.
Maximum Transmission Unit:	The largest packet size a link can carry. Ethernet = 1500B, PPPoE = 1492B, VPN tunnels = 1300-1400B.
Fragmentation:	The process of splitting an oversized datagram into smaller fragments, each with its own IP header copy.
Fragment Offset:	Expressed in 8-byte units. Tells the receiver where each fragment's payload fits in the original datagram.
More Fragments Flag:	MF=1 indicates more fragments follow. MF=0 on the last fragment signals the end of the chain.
Don't Fragment Flag:	DF=1 instructs routers not to fragment. If the packet exceeds MTU, it is dropped.
Identification Field:	A 16-bit value set by the sender to group fragments belonging to the same original datagram.
Reassembly:	The destination host collects fragments (using Identification), orders them (using Offset), and reconstructs the datagram.
ICMP Type 3, Code 4:	"Fragmentation Needed and DF Set" - sent by a router when it cannot fragment a packet with DF=1. Includes Next-Hop MTU.
Path MTU Discovery:	RFC 1191. Uses DF bit + ICMP feedback to find the smallest MTU along a path, avoiding fragmentation entirely.
Header Overhead:	Each fragment carries a copy of the IP header. N fragments produce (N-1) x header_size bytes of overhead.

VIII. IMPLEMENTATION DETAILS

A. Fragmentation Engine

The fragmentation engine (fragmentation.js) is a pure JavaScript module with no external dependencies. It exports six core functions: createPacket, fragmentPacket, fragmentThroughPath, reassembleFragments, simulatePMTUD, and getDemoScenario. The engine handles edge cases including:
- MTU too small for even one payload byte (minimum = header_size + 8)
- Packets that already fit within the MTU (no fragmentation needed)
- DF bit set with oversized packets (ICMP error generation)
- Multi-level fragmentation across multiple links
- Gap detection and reassembly failure on lost fragments

B. State Management

The application uses React hooks (useState, useCallback, useMemo) for state management. The configuration state includes packet parameters, link definitions, and lost fragment selections. All simulation results are derived via useMemo, automatically recomputing when the configuration changes. This reactive approach provides instant visual feedback as users adjust parameters.

C. Visual Design

FragmentLab uses a dark-mode, cyberpunk-inspired design with:
- A deep space color palette with indigo, violet, and cyan accents
- Glassmorphism-style cards with backdrop-filter blur and translucent borders
- Proportional bar charts for packet/fragment size visualization
- SVG-based network path diagrams for PMTUD
- Smooth CSS animations (fadeInUp, slideIn) with staggered delays
- Monospace (JetBrains Mono) typography for numeric and code values
- Status badges with color-coded states (success, error, warning, info)

D. Key Design Decisions

- 8-byte alignment enforcement: Payload sizes are rounded down to the nearest 8-byte multiple using floor((mtu - headerSize) / 8) * 8, matching RFC 791 requirements.
- Dual offset display: The fragment table shows both byte offsets (human-readable) and field values (divided by 8, as stored in the IP header).
- Proportional visualization: Fragment bars are scaled proportionally to packet sizes, providing an intuitive sense of relative sizes.
- Lost fragment simulation: Uses array index tracking rather than modifying fragment objects, allowing instant toggling without re-running the fragmentation engine.

IX. RESULTS & OBSERVATIONS

A. Demo Scenario Results

The default demo scenario (4,000-byte packet, 1,500-byte MTU) produces 3 fragments with total transmitted bytes of 4,040 (1.0% overhead). The fragmentation correctly calculates 1,480-byte payloads for the first two fragments and a 1,020-byte payload for the last fragment.

B. Multi-Hop Scenario Results

Link	MTU (B)	Input Frags	Output Frags	Re-fragmented
Ethernet	1500	1	3	Yes
VPN Tunnel	1300	3	4	Yes
PPPoE	1492	4	4	No

The multi-hop scenario demonstrates cascading fragmentation: the initial 4,000-byte packet is first fragmented into 3 pieces at the Ethernet link (MTU 1500), then re-fragmented to 4 pieces at the VPN Tunnel (MTU 1300). The PPPoE link (MTU 1492) does not cause further fragmentation since all fragments already fit within its MTU. This scenario illustrates how header overhead compounds with each fragmentation stage.

C. Overhead Analysis

Fragmentation overhead grows linearly with the number of fragments. For a 20-byte IP header:
- 3 fragments: +40 bytes overhead (2 extra headers)
- 4 fragments: +60 bytes overhead (3 extra headers)
- 10 fragments: +180 bytes overhead (9 extra headers)

With IP options (60-byte header), overhead increases proportionally. This demonstrates why Path MTU Discovery is preferred in production networks - it eliminates fragmentation overhead entirely.

D. Reassembly Failure Scenario

When Fragment #1 (middle fragment, 1,480 bytes of payload at offset 1480) is marked as lost, the reassembly buffer shows a gap from byte 1480 to byte 2960. The system reports: "Reassembly failed: 1 gap(s) detected in fragment stream. Missing 1480 bytes." This demonstrates that the loss of a single fragment invalidates all other fragments, wasting significant bandwidth.

X. HOW TO RUN

Prerequisites: Node.js (v18+) and npm installed on the system.

```
# Navigate to the project directory
cd ~/Desktop/fragmentlab

# Install dependencies (only needed once)
npm install

# Start the development server
npm run dev

# Open in browser
# http://localhost:5173/
```

The application will start on port 5173 by default. Navigate to http://localhost:5173/ in a modern web browser (Chrome, Firefox, Safari, or Edge). The demo scenario is pre-loaded. Click "Demo" for the single-link scenario or "Multi-Hop" for the multi-link scenario.

XI. CONCLUSION

FragmentLab successfully provides an interactive, educational tool for understanding IPv4 packet fragmentation, reassembly, and Path MTU Discovery. The application implements RFC 791 fragmentation calculations with 8-byte offset alignment, proper MF/DF flag handling, and ICMP message generation. Four distinct modes - Fragmentation, Reassembly, Compare, and PMTUD - cover the complete lifecycle of packet fragmentation from initial split to final reassembly.

The multi-link fragmentation capability demonstrates real-world scenarios where packets traverse paths with varying MTU constraints, and the lost fragment simulation effectively illustrates the fragility of IP-layer fragmentation. The comparison mode clearly shows why modern networks prefer Path MTU Discovery over traditional fragmentation.

Key accomplishments:
- Accurate RFC 791 fragmentation with configurable packet sizes, headers, and MTUs
- Real-time visualization with proportional fragment bars and detailed header field table
- Multi-link cascading fragmentation through heterogeneous network paths
- Interactive reassembly simulation with selectable fragment loss
- Three-way comparison of fragmentation, DF dropping, and PMTUD
- SVG network path diagram with bottleneck detection
- Professional dark-mode UI with glassmorphism design

XII. REFERENCES

[1] J. Postel, "Internet Protocol," RFC 791, DARPA, September 1981.

[2] J. Mogul and S. Deering, "Path MTU Discovery," RFC 1191, IETF, November 1990.

[3] J. Postel, "Internet Control Message Protocol," RFC 792, ISI, September 1981.

[4] A. S. Tanenbaum and D. J. Wetherall, Computer Networks, 5th ed. Pearson, 2011.

[5] J. F. Kurose and K. W. Ross, Computer Networking: A Top-Down Approach, 8th ed. Pearson, 2021.

[6] W. R. Stevens, TCP/IP Illustrated, Volume 1: The Protocols, 2nd ed. Addison-Wesley, 2011.

[7] React Documentation, "React 19," https://react.dev/, accessed October 2026.

[8] Vite Documentation, "Vite 8," https://vite.dev/, accessed October 2026.